COPYRIGHT WARNING: Copyright in these original lectures is owned by Monash University. You may transcribe, take notes, download or stream lectures for the purpose of your research and study only. If used for any other purpose, (excluding exceptions in the Copyright Act 1969 (Cth)) the University may take legal action for infringement of copyright.

Do not share, redistribute, or upload the lecture to a third party without a written permission!

FIT3181/5215 Deep Learning

Week 01: Machine Learning and Mathematical Background Revisit

CE/Lecturer: Dr Trung Le

Lecturer: Dr Binh Nguyen

Email: trunglm@monash.edu



GROUP OF EIGHT AUSTRALIA

Department of Data Science and AI
Faculty of Information Technology, Monash University, Australia

Outline

- Mathematical Background Revisit
- Machine Learning Revisit
- Logistic Regression
- Summary

Mathematical Background Revisited

Linear algebra revision

Why do we need vectors?

Hand-crafted feature

- **Traditional approach** (hand-crafted feature learning)

Input



Feature extraction (SIFT, SURF, HOG)

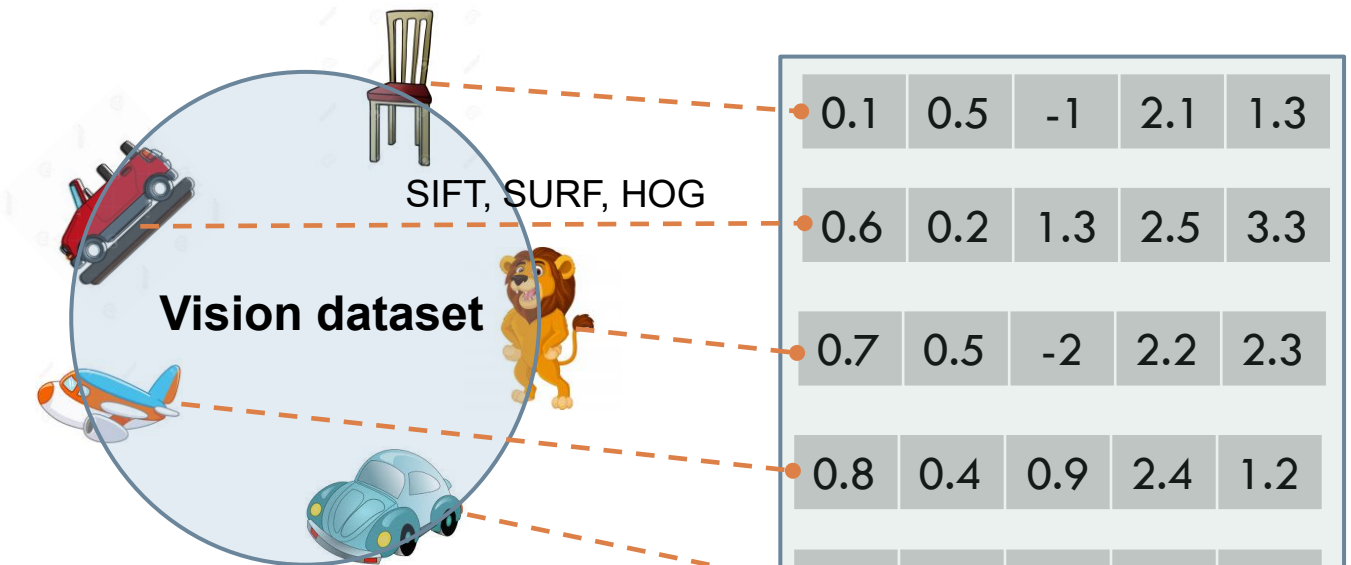
Hand-crafted features



Learning algorithms (SVM, XGBoost)

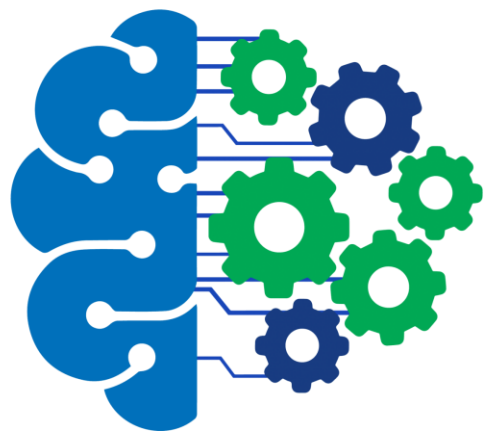


[Source: Internet]



0.1	0.5	-1	2.1	1.3
0.6	0.2	1.3	2.5	3.3
0.7	0.5	-2	2.2	2.3
0.8	0.4	0.9	2.4	1.2
0.6	0.9	1.5	2.0	1.6

Feature vectors



ML model

(SVM, k-NN, decision tree, XG Boost)

Prediction

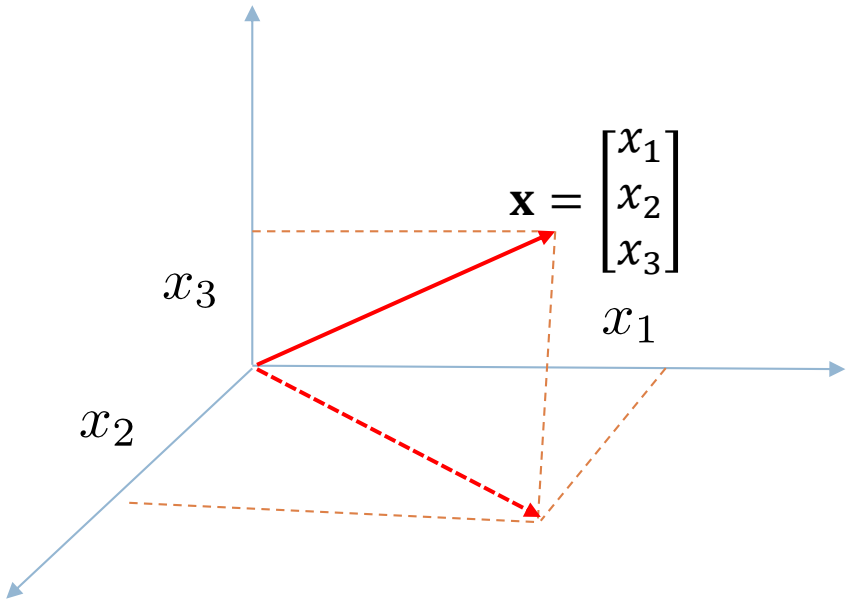
Vector

n-dimensional vector

dimension

$$\mathbf{x} \in \mathbb{R}^n, \ \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

x_i : i-th element



Vector

Operations on vector

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

transpose: column vector to row vector

$$\mathbf{x}^T = [x_1 \; x_2 \; \ldots \; x_n]$$

$$\mathbf{x} + \mathbf{y} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} x_1 + y_1 \\ x_2 + y_2 \\ \vdots \\ x_n + y_n \end{bmatrix}$$

addition

$$\mathbf{x}^T \mathbf{y} = x_1 y_1 + x_2 y_2 + \ldots + x_n y_n$$

inner product

Vector

p-norm

$$\|\mathbf{x}\|_p = (|x_1|^p + |x_2|^p + \cdots + |x_n|^p)^{\frac{1}{p}} \text{ with } p > 0$$

- p = 0 : how many elements in x are non-zeros (sparsity)
- p = 1 : sum of absolute values of elements
- p = 2 : length of the vector

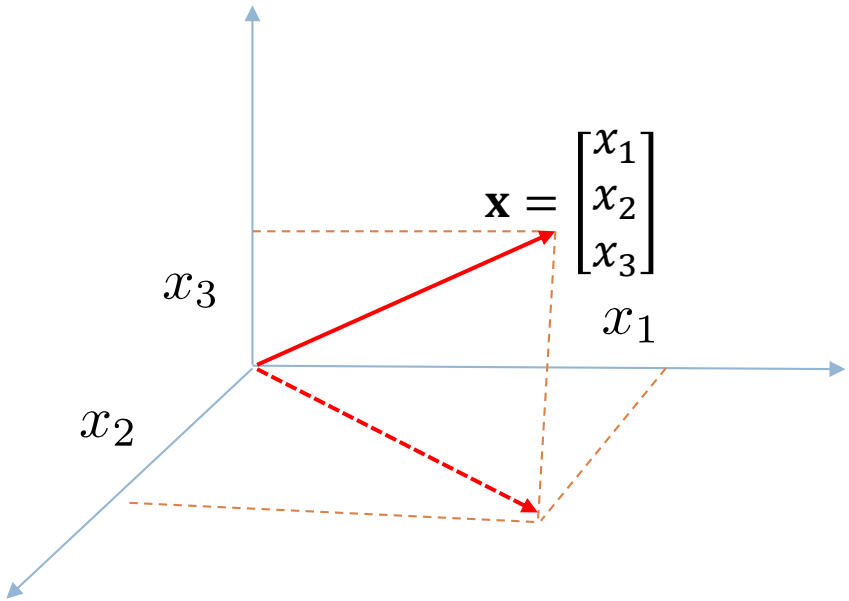
- $\|\mathbf{x}\|_p \geq 0$ and $\|\mathbf{x}\|_p = 0$ if only if $\mathbf{x} = \mathbf{0}$.
- $\|\mathrm{k}\mathbf{x}\|_p = |k|\|\mathbf{x}\|_p$ for any scalar $k \in \mathbb{R}$.
- $\|\mathbf{x} + \boldsymbol{y}\|_p \leq \|\mathbf{x}\|_p + \|\boldsymbol{y}\|_p$ (triangle inequality).

Vector

Length of a vector

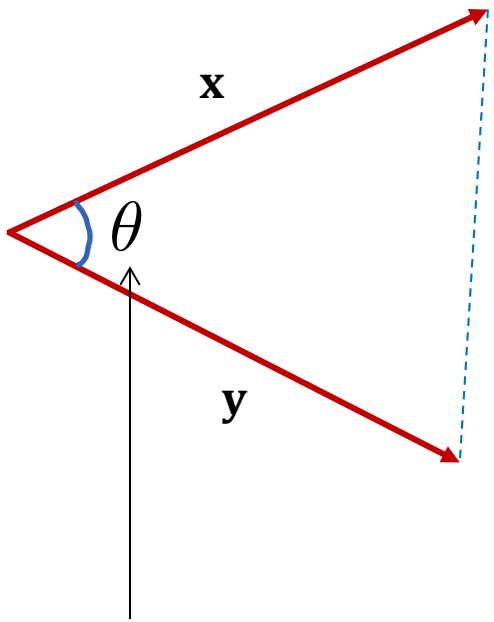
$$\text{length}(\mathbf{x}) = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$

$$\|\mathbf{x}\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$



Vector

Distance between two vectors



Euclidean distance

$$||\mathbf{x} - \boldsymbol{y}|| = \sqrt{(x_1 - y_1)^2 + \cdots + (x_n - y_n)^2}$$

Cosine similarity

$$\cos(\theta) = \cos(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x}^T\mathbf{y}}{||\mathbf{x}||||\mathbf{y}||} = \frac{\mathbf{x}^T\mathbf{y}}{\sqrt{\mathbf{x}^T\mathbf{x}}\sqrt{\mathbf{y}^T\mathbf{y}}}$$

$$-1 \leq \cos(\mathbf{x}, \mathbf{y}) \leq 1$$

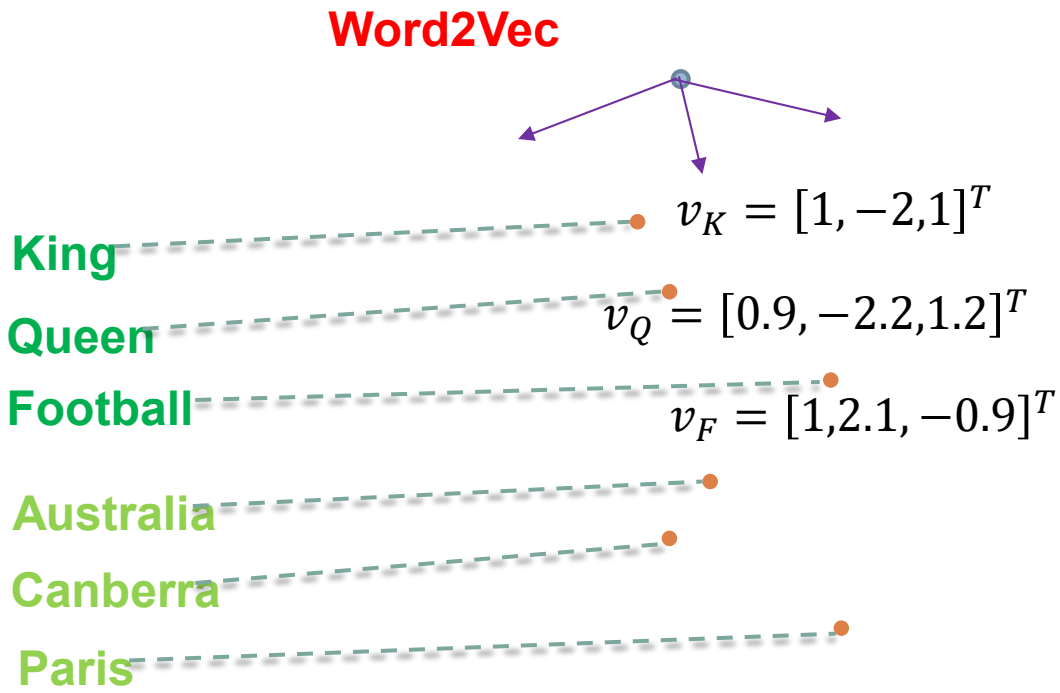
Cosine distance is defined as

$$d_{cosine}(\mathbf{x}, \mathbf{y}) = 1 - \cos(\mathbf{x}, \mathbf{y})$$

Cosine distance can be computed via Euclidean distance if vectors are made unit vectors! (why?)

Example of cosine similarity

- Assume that a **Word2Vec model** transforms three words **King**, **Queen**, and **Football** to three vectors v_K, v_Q, and v_F as shown in the figure. Between two words King and Football, which word is **more similar** to word Queen?



- $cos_sim(King, Queen) = \frac{v_K^T v_Q}{\|v_K\|\|v_Q\|} = \frac{1\times0.9+(-2)\times(-2.2)+1\times1.2}{\sqrt{1^2+(-2)^2+1^2}\sqrt{0.9^2+(-2.2)^2+1.2^2}} = 0.996$
- $cos_sim(Football, Queen) = \frac{v_F^T v_Q}{\|v_F\|\|v_Q\|} = \frac{1\times0.9+2.1\times(-2.2)+(-0.9)\times1.2}{\sqrt{1^2+2.1^2+(-0.9)^2}\sqrt{0.9^2+(-2.2)^2+1.2^2}} = -0.72$
- King is **more similar** to Queen than Football.

Matrix (2D tensor)

number of columns

$$A \in \mathbb{R}^{m \times n}$$

number of rows

number of columns

$$A = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{i1} & \cdots & a_{in} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix}$$

number of rows

$$A = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{i1} & \cdots & a_{in} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix}$$

row vector

column vector

$$A = \begin{bmatrix} a_{11} & \cdots & a_{1j} & \cdots & a_{1n} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ a_{m1} & \cdots & a_{mj} & \cdots & a_{mn} \end{bmatrix}$$

Matrix operation

$$A = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{i1} & \cdots & a_{in} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n}$$

$$B = \begin{bmatrix} b_{11} & \cdots & b_{1n} \\ \vdots & \ddots & \vdots \\ b_{i1} & \cdots & b_{in} \\ \vdots & \ddots & \vdots \\ b_{m1} & \cdots & b_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n}$$

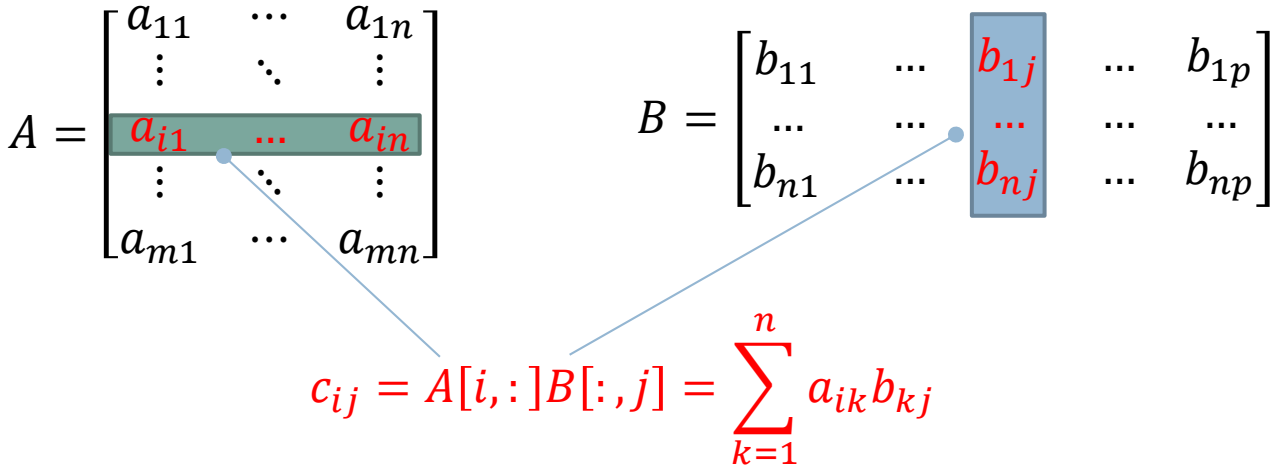
$$C = A^T = \begin{bmatrix} a_{11} & \cdots & a_{m1} \\ \vdots & \ddots & \vdots \\ a_{1j} & \cdots & a_{mj} \\ \vdots & \ddots & \vdots \\ a_{1n} & \cdots & a_{mn} \end{bmatrix} \in \mathbb{R}^{n \times m}$$

$$\alpha A = \begin{bmatrix} \alpha a_{11} & \cdots & \alpha a_{1n} \\ \vdots & \ddots & \vdots \\ \alpha a_{i1} & \cdots & \alpha a_{in} \\ \vdots & \ddots & \vdots \\ \alpha a_{m1} & \cdots & \alpha a_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n}$$

$C = A^T$ if only if $C_{ij} = A_{ji}$ for any $1 \leq i \leq n, 1 \leq j \leq m$

$$C = A + B = \begin{bmatrix} a_{11} + b_{11} & \cdots & a_{1n} + b_{1n} \\ \vdots & \ddots & \vdots \\ a_{i1} + b_{i1} & \cdots & a_{in} + b_{in} \\ \vdots & \ddots & \vdots \\ a_{m1} + b_{m1} & \cdots & a_{mn} + b_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n}$$

Matrix multiplication



- Given **two matrices** $A = [a_{ij}] \in \mathbb{R}^{m \times n}$ and $B = [b_{ij}] \in \mathbb{R}^{n \times p}$, we can compute the **multiplication** $C = AB$ of A and B as
  - $C = [c_{ij}] \in \mathbb{R}^{m \times p}$
  - $c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$ for $1 \leq i \leq m$ and $1 \leq j \leq p$
  - $c_{ij} = A[i,:]B[:,j]$ which is the **dot product** of the **row *i* in A** and the **column *j* in B**.

Matrix multiplication with numpy

```
A = np.array([
    [10,20,30],
    [40,50,60]
])
A
```

```
array([[10, 20, 30],
       [40, 50, 60]])
```

Declare matrix A with the size 2×3

```
D = np.array([
        [ 2,  3,  5,  7],
        [11, 13, 17, 19],
        [23, 29, 31, 37]
    ])
D
```

```
array([[ 2,  3,  5,  7],
       [11, 13, 17, 19],
       [23, 29, 31, 37]])
```

Declare matrix D with the size 3×4

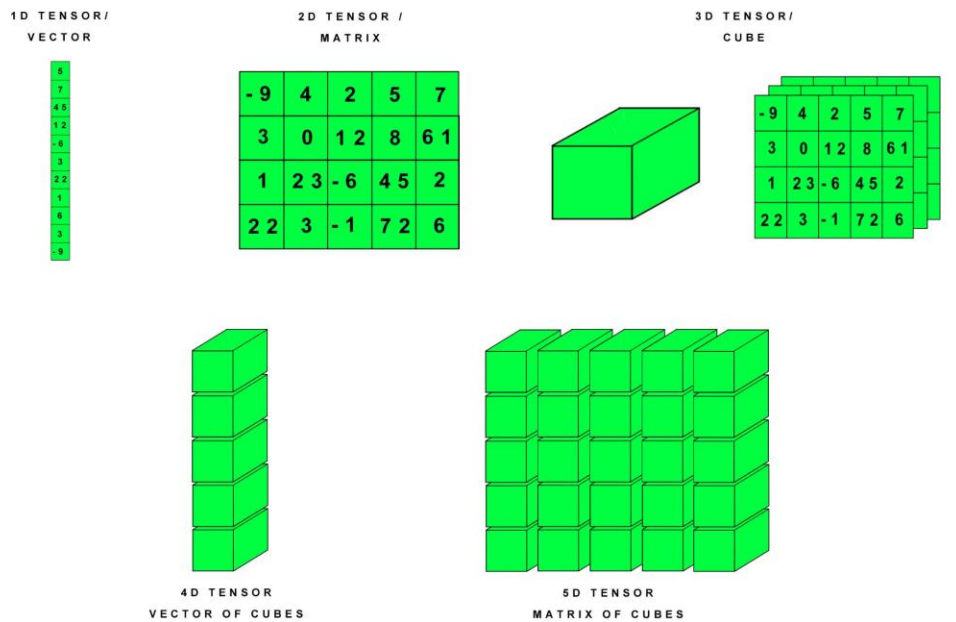
```
E = A.dot(D)
E
```

```
array([[ 930, 1160, 1320, 1560],
       [2010, 2510, 2910, 3450]])
```

$E = A \times D$ has the size 2×4

Tensor

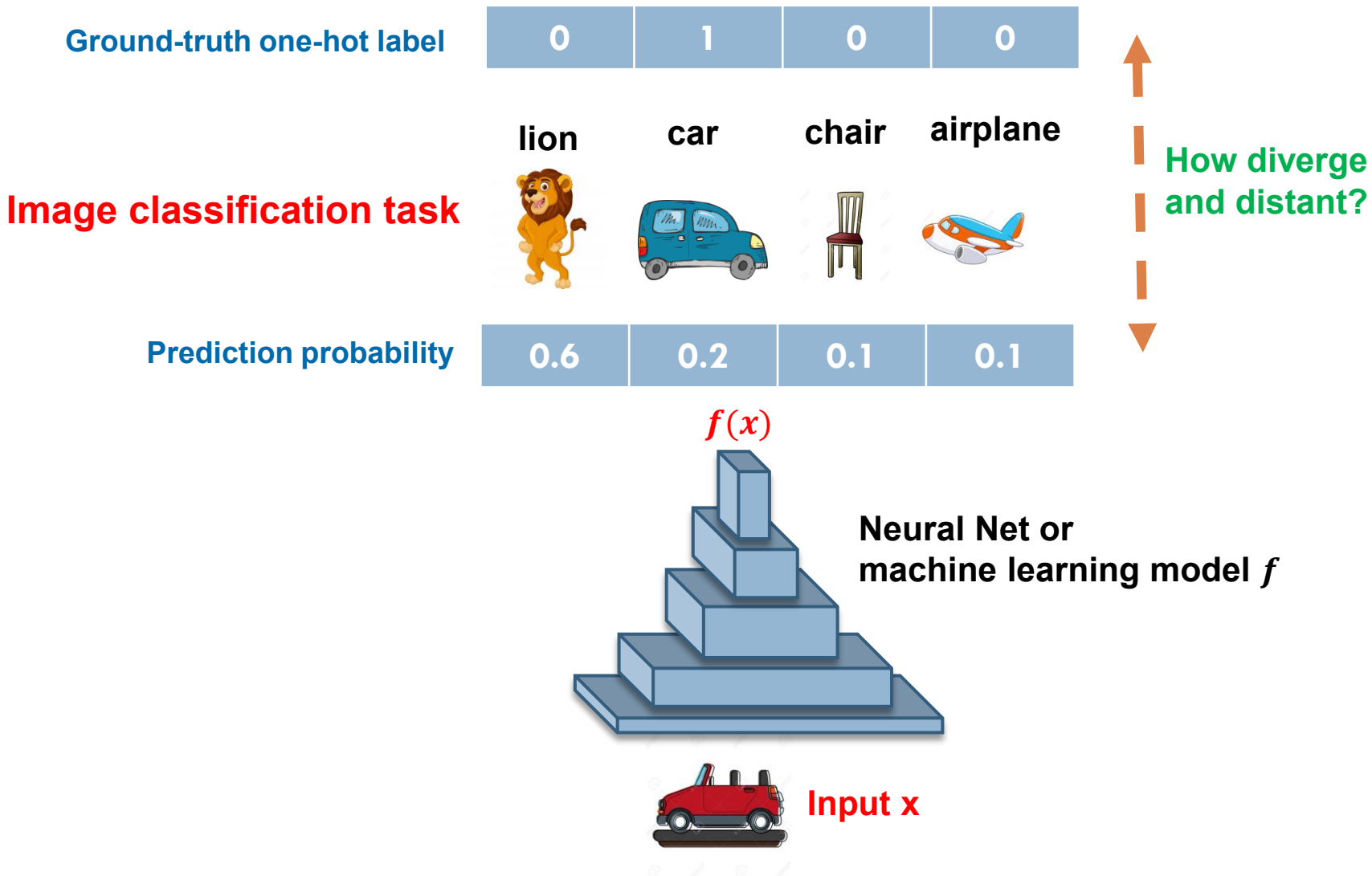
- A **tensor** is simply a **multidimensional** array
  - 0D tensor: a scalar
  - 1D tensor: a vector
  - 2D tensor: a matrix
  - 3D tensor: a 3D hyperrectangle of numerals



Examples of tensors. [Source: Internet]

Information theory revision

Why need to work with discrete distributions?



Revision of basic informatic quantities

- Given two **discrete distributions** $p = [p_i]_{i=1}^d$ ($p_i \geq 0$ and $\sum_{i=1}^d p_i = 1$) and $q = [q_i]_{i=1}^d$ ($q_i \geq 0$ and $\sum_{i=1}^d q_i = 1$).

- **Kullback-Leibler (KL) divergence between p, q**
  - $KL(p,q) = \sum_{i=1}^d p_i \log \frac{p_i}{q_i}$
  - $KL(p,q) \geq 0$ and $KL(p,q) = 0$ if only if $p = q$

- **Entropy of the distribution p**
  - $H(p) = \sum_{i=1}^d p_i \log \frac{1}{p_i} = -\sum_{i=1}^d p_i \log p_i$ where $0 \times \log 0 = 0$

- **Cross-entropy (CE) divergence between p, q**
  - $CE(p,q) = -\sum_{i=1}^d p_i \log q_i$.
  - $CE(p,q) = KL(p,q) + H(p)$.

Example

- Consider $p = [0.1, 0.4, 0.5]$ and q $= [0.2, 0.5, 0.3]$. Compute $KL(p, q)$, $CE(p, q)$, and $H(p)$.
  - $KL(p,q) = 0.1 \times \log\frac{0.1}{0.2} + 0.4 \times \log\frac{0.4}{0.5} + 0.5 \times \log\frac{0.5}{0.3} \approx 0.042$
  - $CE(p,q) = -0.1 \times \log 0.2 - 0.4 \times \log 0.5 - 0.5 \times \log 0.3 \approx 0.452$
  - $H(p) = 0.1 \times \log\frac{1}{0.1} + 0.4 \times \log\frac{1}{0.4} + 0.5 \times \log\frac{1}{0.5} \approx 0.41$

- Consider $p = [0,1,0]$ and q $= [0.2,0.3,0.5]$. Compute $CE(p, q)$.
  - $CE(p,q) = -0 \times \log 0.2 - 1 \times \log 0.3 - 0 \times \log 0.5 \approx -\log 0.3 = 0.523$

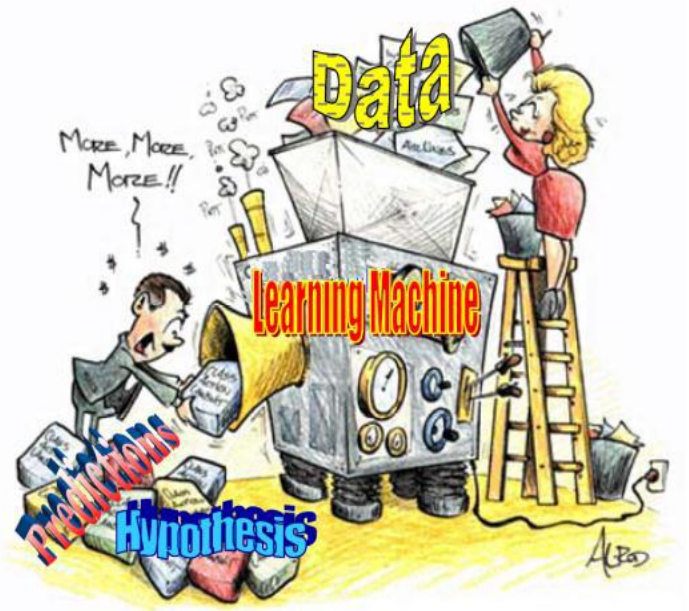
Question: when the entropy is maximized or minimized?


Q&A
WHO?
WHERE?
WHEN?
WHAT?
How?
Why?
When?
Where?
HOW?
What?

Machine Learning Revisited

Machine learning

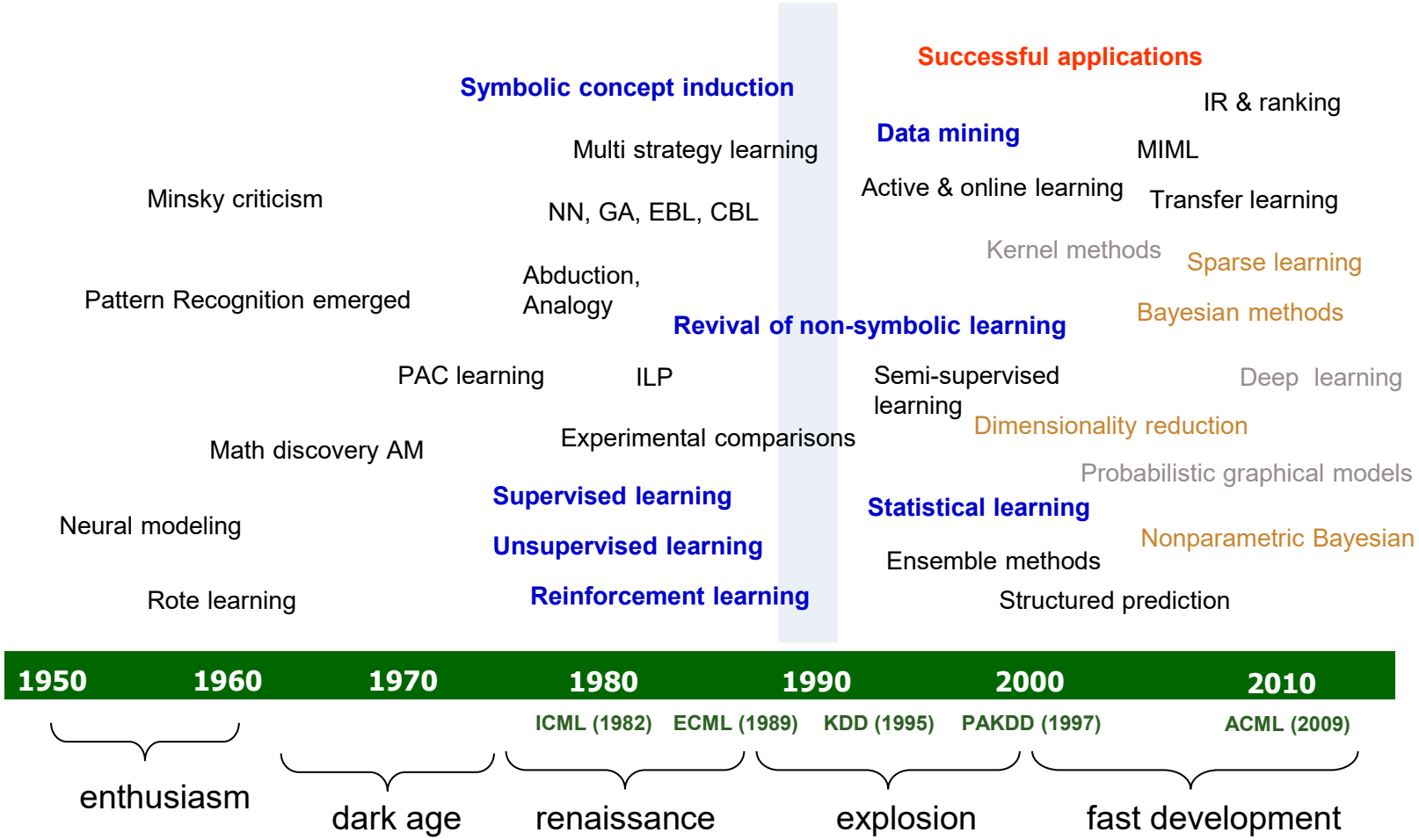
- "Field of study that gives computers the ability to learn without being explicitly programmed" (Arthur Samuel, 1959).
- "Machine learning sits at the crossroads of computer science, statistics and a variety of other disciplines concerned with automatic improvement over time, and inference and decision-making under uncertainty." (Jordan & Michell, 2015).
- *"A computer program is said to learn from **experience** E with respect to some **class of tasks** T and **performance measure** P, if its **performance at tasks** in T, as **measured** by P, improves with **experience** E"*, (Tom Mitchell, 1997)



(from Eric Xing lecture notes)

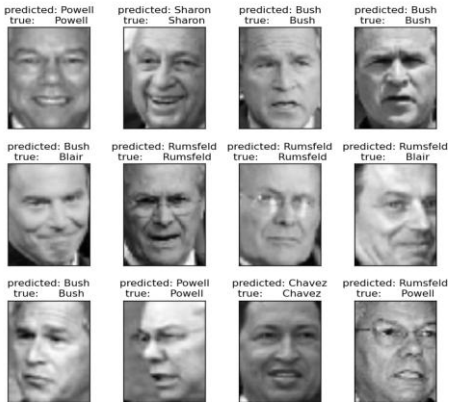
M.I. Jordan and T. Mitchell, "Machine Learning: Trends, perspectives, and prospects", *Science*, 17 July 2015.

A brief history of ML



[credit: Prof Bao Ho, JAIST]

Machine learning



Face recognition system

Experience E	Task T	Performance P
dataset of thousands of known faces	given a new photo, recognise the name of the face **Label set**: names of all people.	how **accurate** the recognition is Measure: **Accuracy**





Detecting Spam Emails

Experience E	Task T	Performance P
Examples of spam emails and not-spam emails	To assign a label "spam" or "not-spam" to an email Label set: **spam** and **not spam**	how **accurate** spam email can be detected Measure: **accuracy**



Detect credit card fraud

Experience E	Task T	Performance P
Data collected for credit-card transactions deemed as fraud and not-fraud	To assign a label 'fraud' or "not fraud" to a given credit-card transaction Label set: **fraud** and **not fraud**	how accurate a credit-card fraud transaction can be detected. Measure: **accuracy**

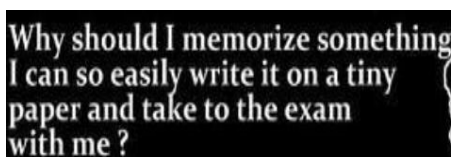
Elements of machine learning

1 Data

Vector $\boldsymbol{x} \in \mathbb{R}^d$
Image $\boldsymbol{x}$
Sentence $\boldsymbol{x}$
Speech $\boldsymbol{x}$
Ground-truth label $y \in Y$

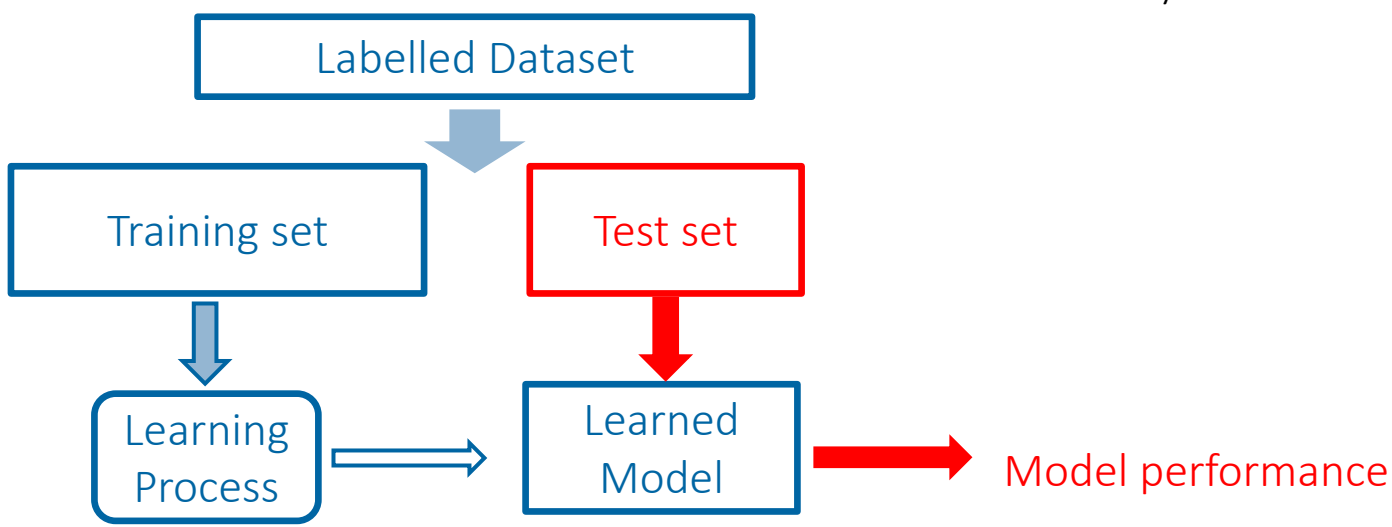






2 Model

- $f: X \rightarrow Y$
- X is **data space**, Y is **label space**
- **Classification**: $Y = \{1,2, \dots, M\}$
- **Regression**: $Y = \mathbb{R}$
- **Prediction**: $f(x) = \hat{y} \in Y$



Machine learning pipeline.

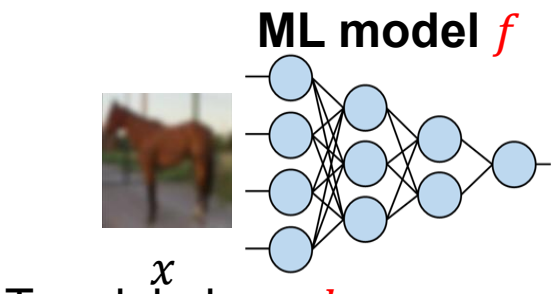
3 Assessment

- How **good** is the model doing its tasks?
- **Performance metrics**
  - Accuracy, Recall, Precision, F-score
  - $Accuracy = \frac{\#Correct\ Predictions}{\#All\ Predictions}$
    : % of correctly classified examples

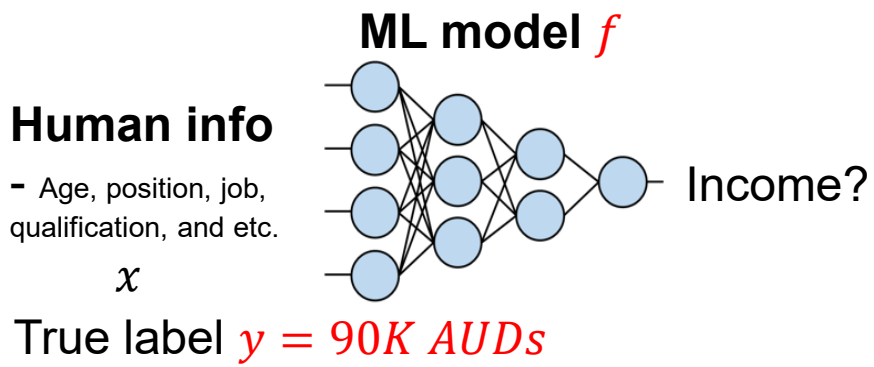
Classification and Regression

- Two **main tasks** in machine learning
  1. **Classification** and **regression**
  2. Other tasks in ML include density estimation, clustering, anomaly detection, recommendation, forecasting, and ranking

- **Classification**
  1. The label set $Y = \{1,2, \dots, M\}$ and the data space X (vectors, images).
  2. Training set $\boldsymbol{D} = \{(\boldsymbol{x_1}, y_1), \dots, (\boldsymbol{x_N}, y_N)\}$ where $\boldsymbol{x_i} \in \mathbb{R}^d$ and $y_i \in Y = \{1,2, \dots, M\}$
  3. Learn a model $f: X \to Y$
  4. Given data example $\boldsymbol{x}$, render the prediction $\hat{y} = f(\boldsymbol{x}) \in \{1,2, \dots, M\}$



- **Regression**
  1. The label set $Y = \mathbb{R}$ and the data space X (vectors, images).
  2. Training set $\boldsymbol{D} = \{(\boldsymbol{x_1}, y_1), \dots, (\boldsymbol{x_N}, y_N)\}$ where $\boldsymbol{x_i} \in \mathbb{R}^d$ and $y_i \in Y = \mathbb{R}$
  3. Learn a model $f: X \to Y$
  4. Given data example $\boldsymbol{x}$, render the prediction $\hat{y} = f(\boldsymbol{x}) \in \mathbb{R}$



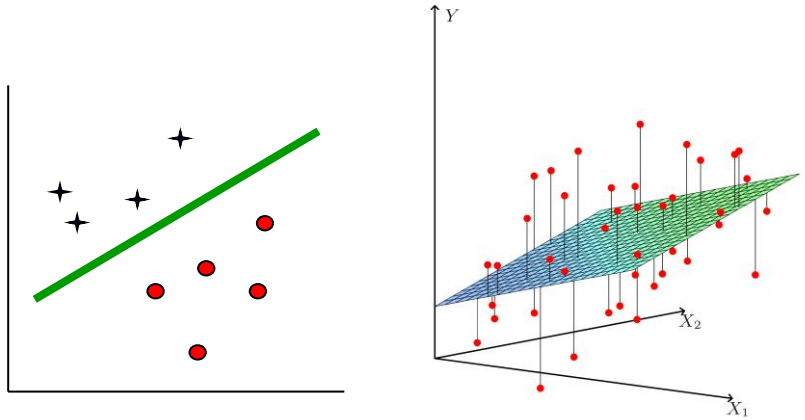
Supervised learning

Problem: Learn a **function** from **data** to **relate the inputs** with **outputs**. Training data include output information (labels).

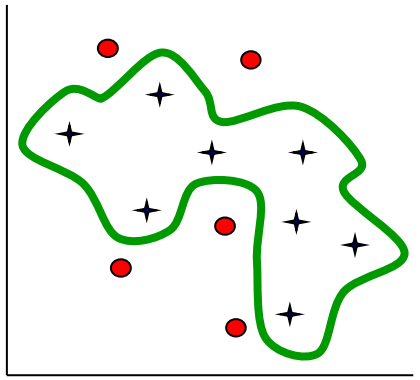
Data: $D = \{(\boldsymbol{x_1}, y_1), \ldots, (\boldsymbol{x_N}, y_N)\}$

Function: $f: X \rightarrow Y$

$x \in \mathbb{R}^d$ = feature
$y \in \{1,2, \ldots, M\}$ = a discrete label (classification),
$y \in \mathbb{R}$ = a continuous value (regression)



e.g., linear functions

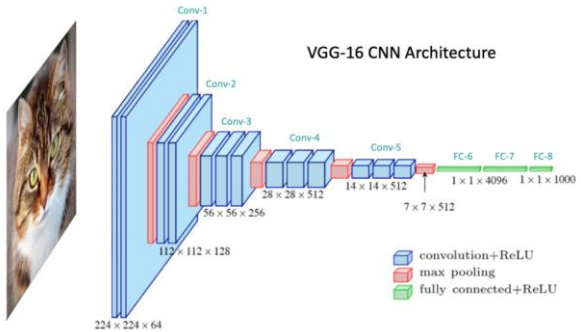


e.g., nonlinear functions

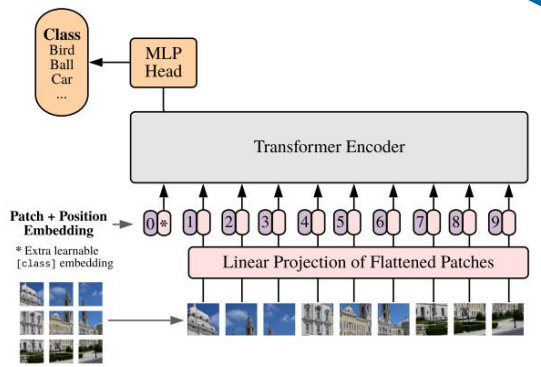
Supervised learning



Labelled vision dataset



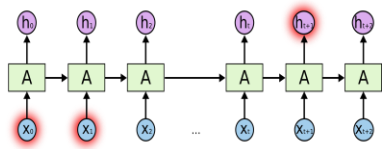
Convolution Neural Network



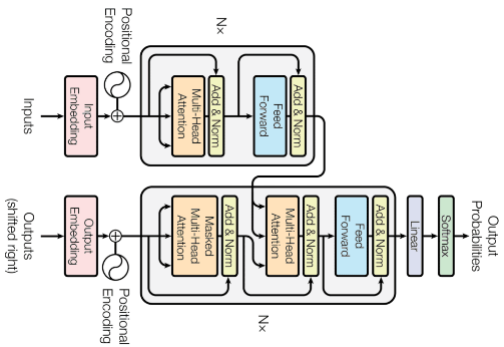
Vision Transformers

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive
5	Probably my all-time favorite movie, a story o...	positive
6	I sure would like to see a resurrection of a u...	positive
7	This show was an amazing, fresh & innovative i...	negative
8	Encouraged by the positive comments about this...	negative
9	If you like original gut wrenching laughter yo...	positive

Labelled text corpus



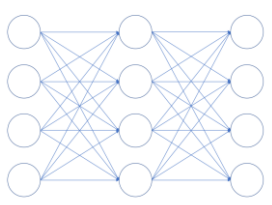
Recurrent neural nets



Transformers

PREG	PLAS	BP	SKIN	INS	BMI	PED	AGE	DIABETES
6	148	72	35	148	33.6	0.627	50	Y
1	85	66	29	85	26.6	0.351	31	N
8	183	64	23.3	183	23.3	0.672	32	Y
1	89	66	23	94	28.1	0.167	21	N
5	116	74	25.6	116	25.6	0.201	30	N
3	78	50	32	88	31	0.248	26	Y
10	115	35.3	35.3	115	35.3	0.134	29	N
2	197	70	45	543	30.5	0.158	53	Y
8	125	96	54	125	54	0.232	54	Y
4	110	92	37.6	110	37.6	0.191	30	N
10	168	74	38	168	38	0.537	34	Y
10	139	80	27.1	139	27.1	1.441	57	N
1	189	60	23	846	30.1	0.398	59	Y

Labelled tabular dataset

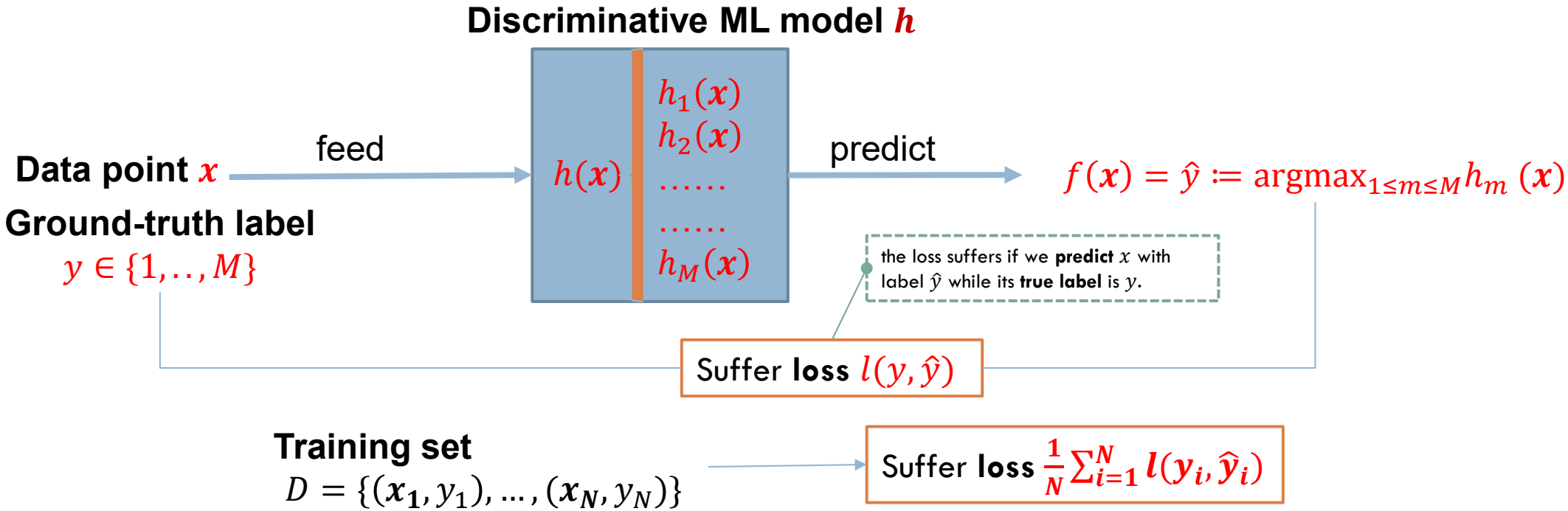


Multilayered feedforward nets

Discriminative Machine Learning and Logistic Regression

Discriminative machine learning

Classification

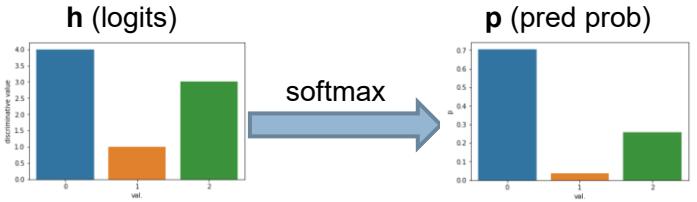


- A data point x receives **discriminative values/logits** $h_1(x), \dots, h_M(x) \in \mathbb{R}$ from the model
  - $h_m(x)$ represents the **possibility** to **classify** x to **class** m for $1 \le m \le M$
- We use these **discriminative values/logits** to predict the label $\hat{y}$ as
  - $\hat{y} = \text{argmax}_{1 \le m \le M} h_m(x)$, meaning the class with **highest discriminative value/logit**
- The prediction x with the predicted label $\hat{y}$ suffers a loss
  - $l(\hat{y}, y)$ where l is a **loss function** (if $\hat{y} = y$ then $l(\hat{y}, y) = 0$).
- Given a training set $D = \{(x_1, y_1), \dots, (x_N, y_N)\}$, the loss incurred is
  - $\frac{1}{N}\sum_{i=1}^{N} l(y_i, \hat{y}_i)$

Softmax activation function

- Transform **real-valued discriminative scores** to **discrete probabilities**
  - $h = [h_m]_{m=1}^{M} \rightarrow p = \text{softmax}(h) := \left[\frac{\exp\{h_m\}}{\sum_{i=1}^{M}\exp\{h_i\}}\right]_{m=1}^{M}$
  - $p = [p_m]_{m=1}^{M}$ becomes a distribution over classes $\{1, \dots, M\}$
    - $p_m \geq 0$ $(1 \leq m \leq M)$ and $\sum_{m=1}^{M} p_m = 1$.

- Assume that we do **sentiment classification** (analysis)
  - Data point x is a sentence (e.g., $x =$ "$it\ is\ a\ beautiful\ day\ today$"), we need to predict **sentiment** of this sentence
  - Three **sentiment labels**: positive (happy, class 1), negative (sad, class 2), neutral (none happy and sad, class 3)
  - The model gives three **discriminative values** for x
    - $h_1 = 2, h_2 = -1, h_3 = 1$ means that **highest possibility** to **classify** x to the **class 1** with the label **positive**.
  - We apply **softmax** function on the **discriminative scores** h
    - $p = \text{softmax}(h)$
      - $p_1 = \frac{\exp\{2\}}{\exp\{2\}+\exp\{-1\}+\exp\{1\}} \approx 0.705$, $p_2 = \frac{\exp\{-1\}}{\exp\{2\}+\exp\{-1\}+\exp\{1\}} \approx 0.035$, $p_3 = \frac{\exp\{1\}}{\exp\{2\}+\exp\{-1\}+\exp\{1\}} \approx 0.259$
    - $p = [0.70538451, 0.03511903, 0.25949646]$ are the **probabilities** to **classify** x to the **classes 1, 2, 3** respectively.



Softmax *transforms real-valued discriminative scores (ranged in* $(-\infty, +\infty)$*) to probability values (ranged in* $[0, 1]$*) but preserving the order.*

Cross-entropy loss

- Given M labels $1,2,\dots,M$, a label $y \in \{1,\dots,M\}$ has two equivalent forms:
  - **Numeric form**: $y \in \{1,\dots,M\}$ is an **index** in $\{1,\dots,M\}$.
  - **One-hot vector form**: $1_y = [\underbrace{0,..,0,1,0,\dots,0}_{y\text{-th position}}]$- the vector of **zeros** except **a unique 1** in the y-th position.
- Label set $\{happy, sad, neural\}$ with $1: happy, 2: sad,$ and $3: neutral$
  - **[1,0,0]** → happy (numeric label 1), **[0,1,0]**→sad (numeric label 2), **[0,0,1]**→neural (numeric label 3)
- Given two **discrete distributions** over classes: $p = [p_m]_{m=1}^M$ and $\mathrm{q} = [q_m]_{m=1}^M$, the **cross-entropy divergence** between p and q
  - $\mathrm{CE}(p,q) := -\sum_{m=1}^M p_m \log q_m$ which measures how far (divergent) it is between p and q.
  - $\mathrm{CE}(p,q) \geq \mathrm{H}(p) := -\sum_{m=1}^M p_m \log p_m$ (H is the **Shannon entropy**).

Cross-entropy loss

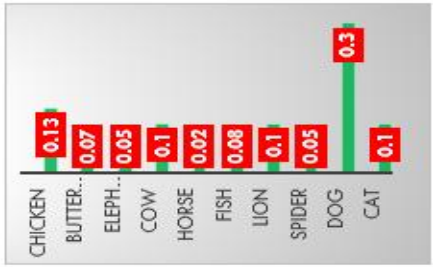
- **Cross-entropy loss**
  - $l(y, \hat{y}) = \mathrm{CE}(1_y, p) = -\log p_y$
  - $y \in \{1, \dots, M\}$ is the **ground-truth label** of a given x and p is **the prediction probabilities** of x to classes

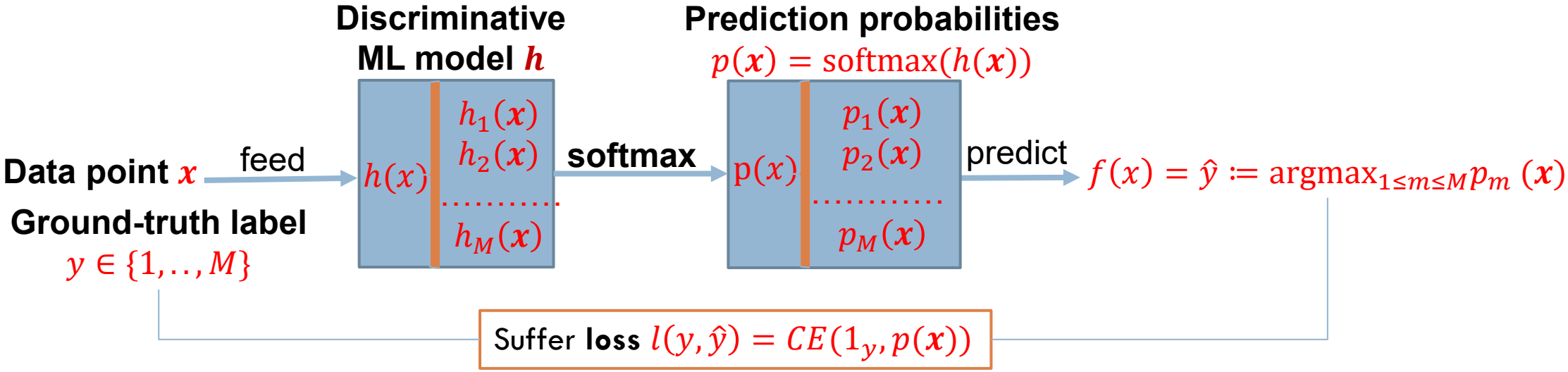
- Given **a sentence** x with the **label positive/happy** (the class 1), assume that our model predicts it with **prediction probabilities** $p = [0.3, 0.4, 0.3]$, the **cross-entropy loss** for this prediction
  - $l(y, \hat{y}) = CE([1,0,0], [0.3,0.4,0.3]) = -1.\log 0.3 - 0.\log 0.4 - 0.\log 0.3 = -\log 0.3 \approx 1.204$
  - **Correct** or **Incorrect** prediction?

Question: why is the **cross-entropy loss** always **non-negative** and when it is **0**?

Discriminative machine learning

Classification with the cross-entropy loss

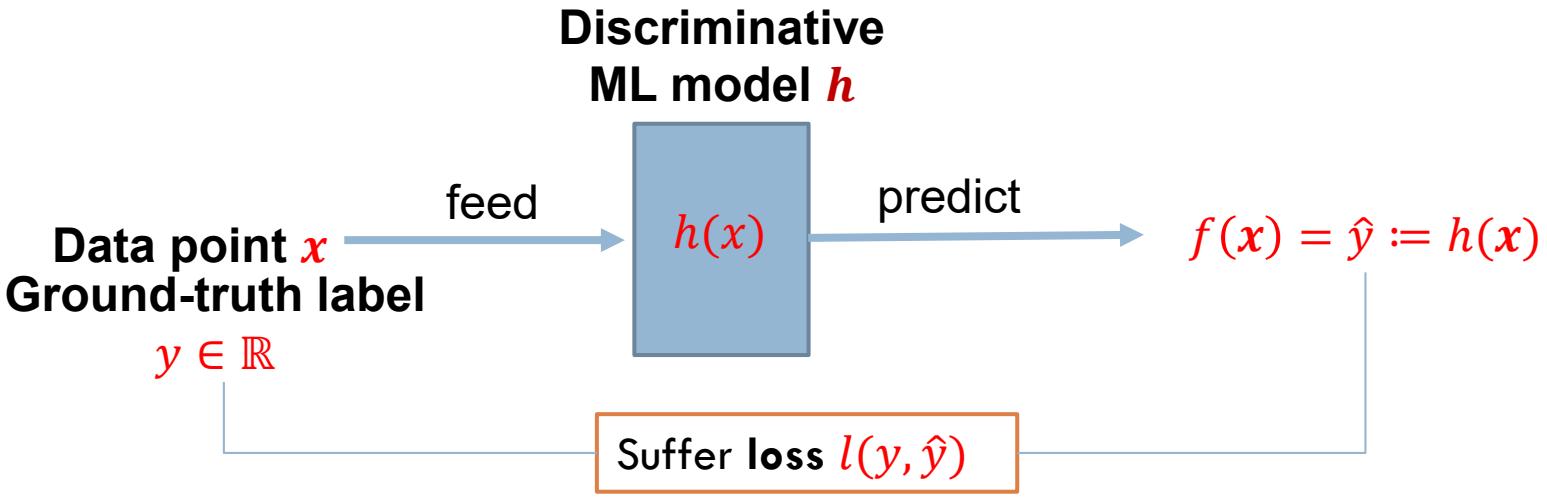




- We apply the **softmax function**: $\mathbf{p(x) = softmax(h(x))}$ to make $p(x)$ **prediction probabilities**
  - $p_m(x) = p(y = m|x)$ is the **probability** to classify x to the class m for $1 \le m \le M$
- The loss incurred for this prediction
  - $l(y, \hat{y}) = CE\left(1_y, p(x)\right) = -\log p_y(x)$ (**negative log likelihood**)
- The **loss for the training set** $D = \{(x_1, y_1), \dots, (x_N, y_N)\}$
  - $L(D, h) = \frac{1}{N}\sum_{i=1}^{N} CE(1_{y_i}, p(x_i)) = -\frac{1}{N}\sum_{i=1}^{N} \log p_{y_i}(x_i)$

Discriminative machine learning

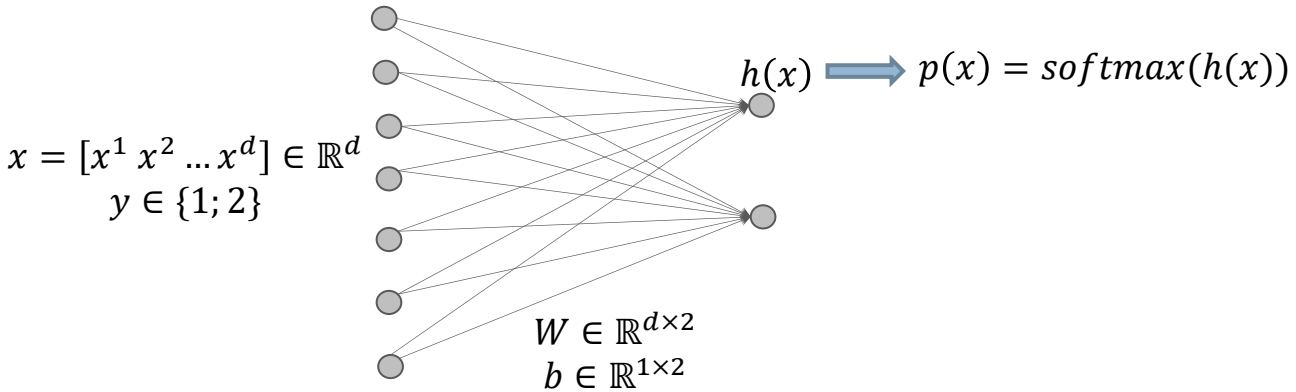
Regression



- Only **one discriminative value** $h(\boldsymbol{x}) \in \mathbb{R}$, we use this to predict $\hat{y} = f(\boldsymbol{x}) \coloneqq h(\boldsymbol{x})$.
- The **potential losses** could be
  - **L2 loss**: $l(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$
  - **L1 loss**: $l(y, \hat{y}) = |y - \hat{y}|$
  - **ϵ-insensitive loss**: $l(y, \hat{y}) = \max\{0, |y - \hat{y}| - \epsilon\}$ where $\epsilon \geq 0$.

Logistic regression

A simple feed-forward neural network

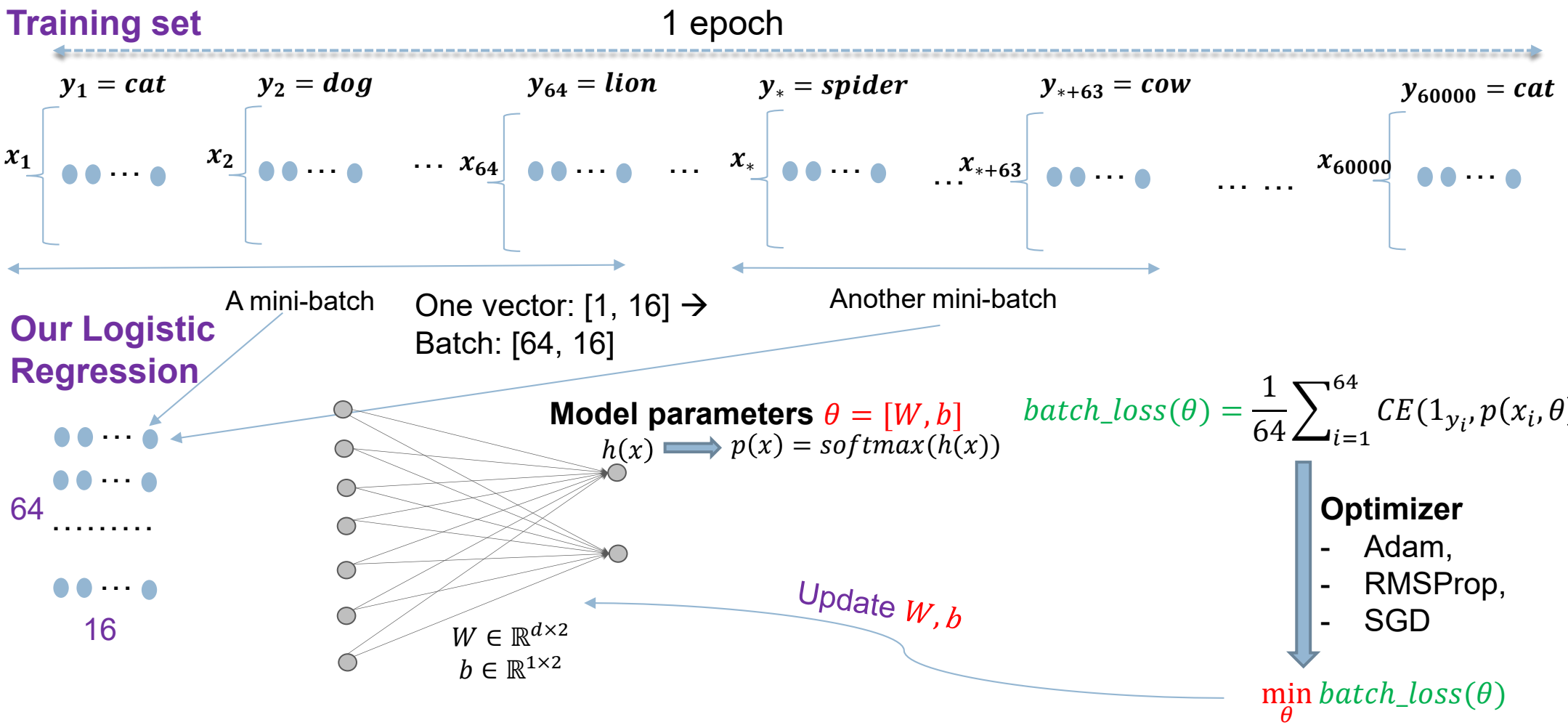


- **Computational process**
  - $h(x) = [h_1(x)\ \ h_2(x)] = xW + b$
    - $h(x) \in \mathbb{R}^{1\times 2}$ contains **discriminative scores** with respect to classes 1 and 2.
  - $p(x) = [p_1(x)\ p_2(x)] = softmax(h(x))$
    - $p(x)$ is the **prediction probabilities** with respect to classes 1 and 2.
  - $\hat{y} = \begin{cases} \mathbf{1}, & \boldsymbol{p_1(x) \geq p_2(x)} \\ \mathbf{2}, & \boldsymbol{p_1(x) < p_2(x)} \end{cases}$
  - $l(y, \hat{y}) = CE\left(1_y, p(x)\right) = -\log p_y\,(x)$ → the **cross-entropy loss**.
- **Training process**
  - **Training set** $D = \{(\boldsymbol{x_1}, y_1), \dots, (\boldsymbol{x_N}, y_N)\}$
  - $L(D; W, b) = \frac{1}{N}\sum_{i=1}^{N} CE(1_{y_i}, p(\boldsymbol{x_i})) = -\frac{1}{N}\sum_{i=1}^{N} \log p_{y_i}(\boldsymbol{x_i})$
  - $\min_{W,b} L(D; W, b)$ → $(W^*, b^*) = \text{argmin}_{W,b} L(D; W, b)$

TorchOpt

```
"Adam":torch.optim.Adam
"Adadelta":torch.optim.Adadelta
"Adagrad":torch.optim.Adagrad
"Adamax":torch.optim.Adamax
"AdamW": torch.optim.AdamW
"ASGD":torhc.optim.ASGD,
"NAdam":torch.optim.NAdam,
"RMSprop":torhc.optim.RMSprop
"RAdam":torch.optim.RAdam
"Rprop": torch.optim.Rprop
"SGD":torch.optim.SGD
```

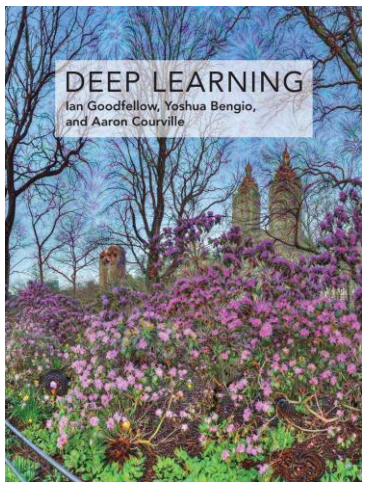
Logistic Regression Pipeline

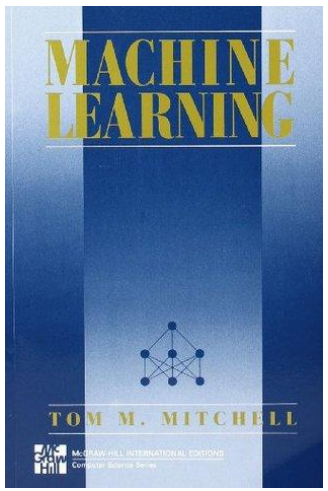


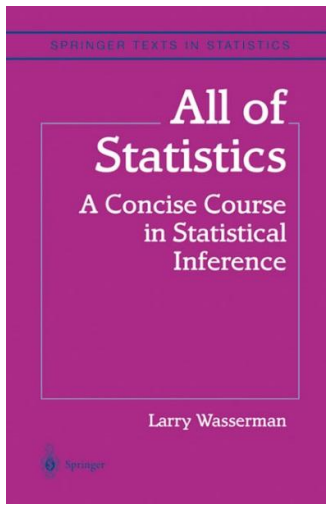

Q&A
WHO?
WHERE?
WHEN?
WHAT?
How?
Why?
When?
Where?
HOW?
What?

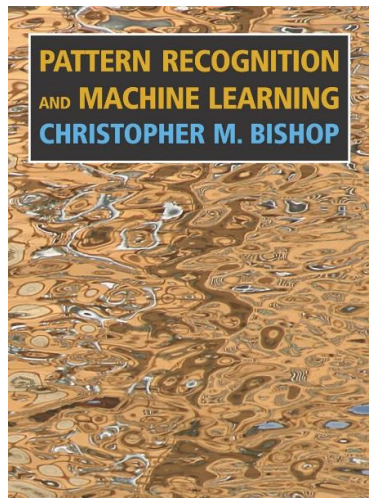
Appendix

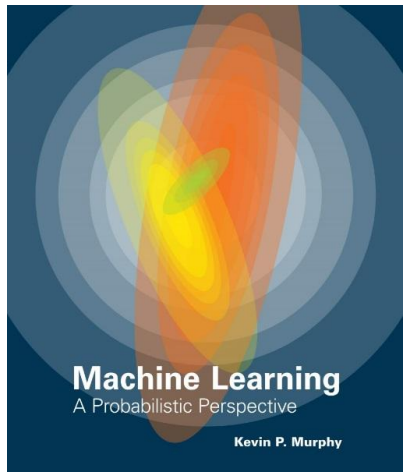
Other ML Books

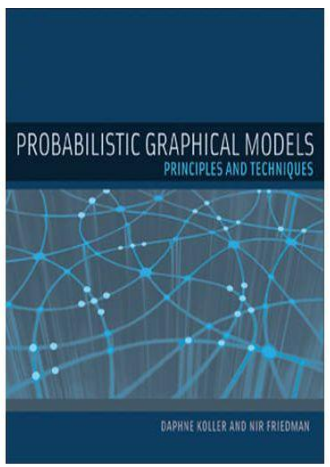


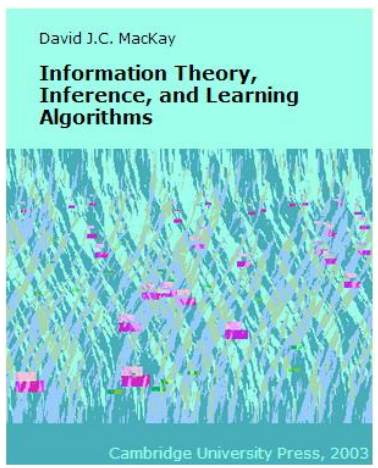


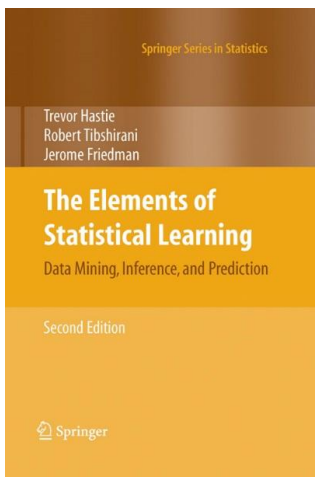


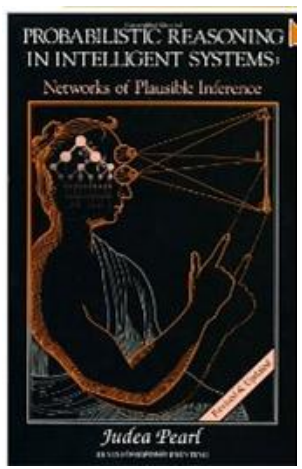


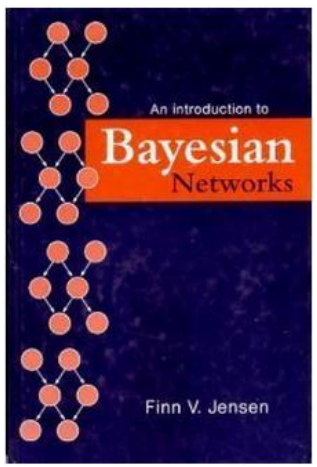












Machine learning: five tribes

Tribes	Origins	Master Algorithm
Symbolists	Logic, philosophy	*Inverse deduction*

The Symbolist School



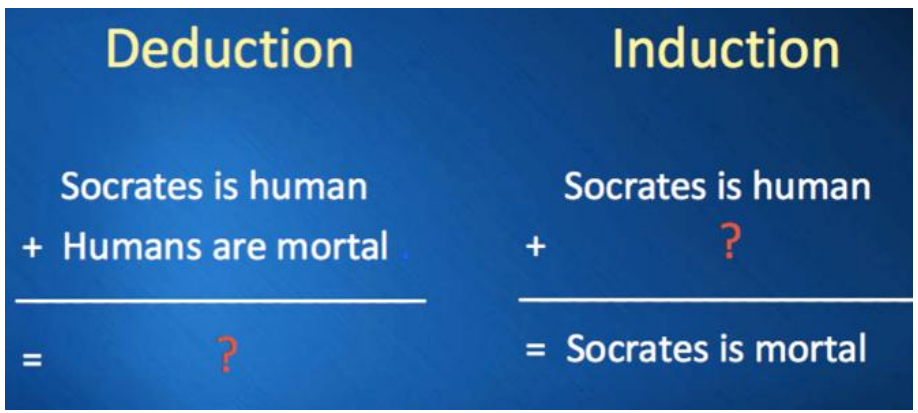
Tom Mitchell



Steve Muggleton



Ross Quinlan



Decision trees, C4.5, first-order propositional logic, rule-based reasoning systems, etc.

Machine learning: five tribes

Tribes	Origins	Master Algorithm
Symbolists	Logic, philosophy	*Inverse deduction*
Evolutionary	Evolutionary biology	*Genetic programming*

The Evolutionary School



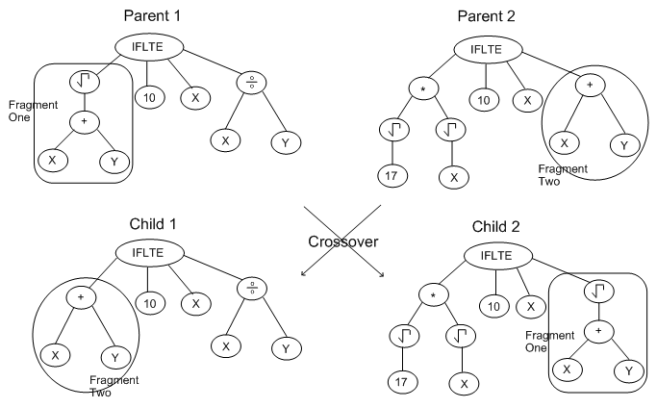
John Koza



John Holland



Hod Lipson



Genetic programming, genetic algorithms, fuzzy systems

Machine learning: five tribes

Tribes	Origins	Master Algorithm
Symbolists	Logic, philosophy	*Inverse deduction*
Evolutionary	Evolutionary biology	*Genetic programming*
Connectionists	Neuroscience	*Backpropagation*

The Connectionist



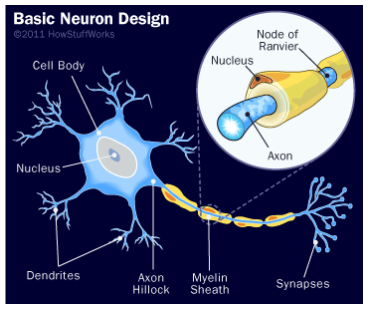
Geoff Hinton

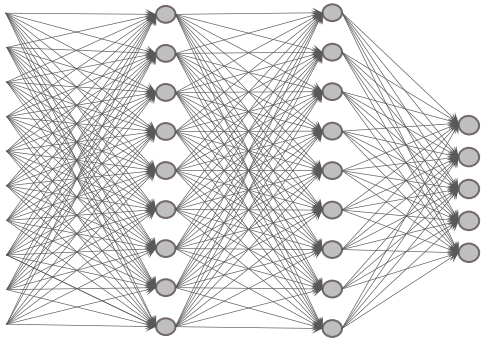


Yann LeCun



Yoshua Bengio



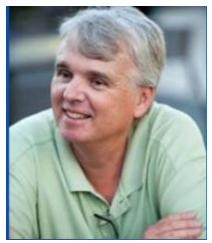


Deep Neural Networks, Convolutional Neural Networks, Deep learning, etc.

Machine learning: five tribes

Tribes	Origins	Master Algorithm
Symbolists	Logic, philosophy	*Inverse deduction*
Evolutionary	Evolutionary biology	*Genetic programming*
Connectionists	Neuroscience	*Backpropagation*
Bayesians	Statistics	*Bayes' theorem*

The Bayesians



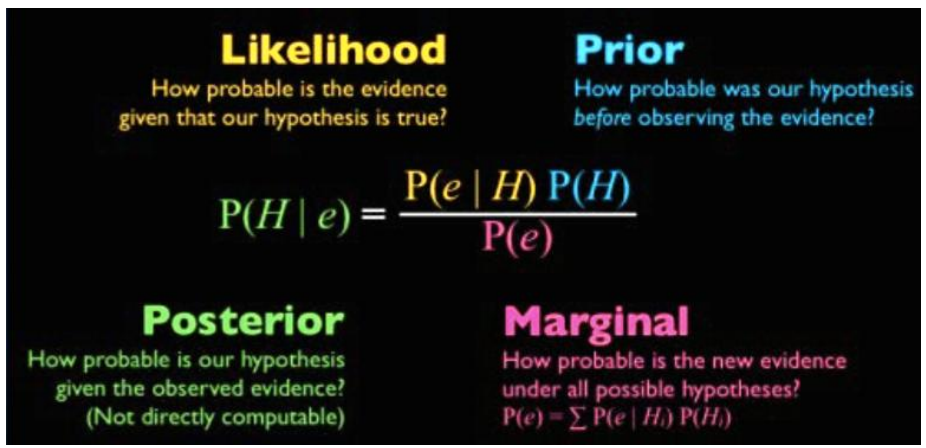
David Heckerman



Judea Pearl



Michael Jordan



Bayesian networks classifiers, graphical models, Hidden Markov models, Conditional Random Fields, most of statistical machine learning approaches …

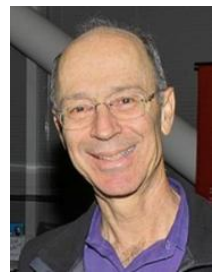
Machine learning: five tribes

Tribes	Origins	Master Algorithm
Symbolists	Logic, philosophy	*Inverse deduction*
Evolutionary	Evolutionary biology	*Genetic programming*
Connectionists	Neuroscience	*Backpropagation*
Bayesian	Statistics	*Bayes' theorem*
Analogizers	Psychology	*Support Vector Machines*

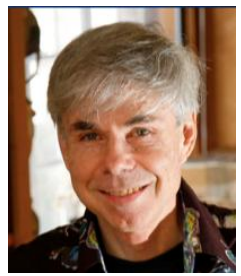
The Analogizers



Vladimir Vapnik

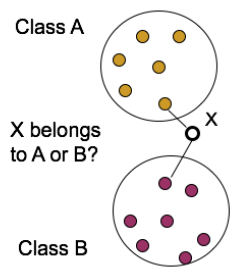


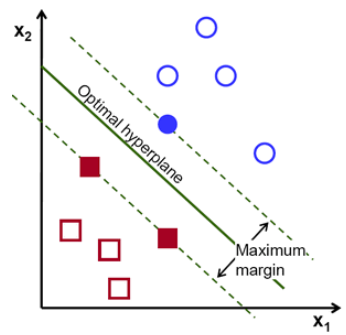
Peter Hart



Douglas Hofstadter

- **Instance-based classification**
  - Using most similar individual instances known in the past to classify a new instance
- **Typical approaches**
  - **k-nearest neighbor approach**
    - Instances represented as points in a Euclidean space



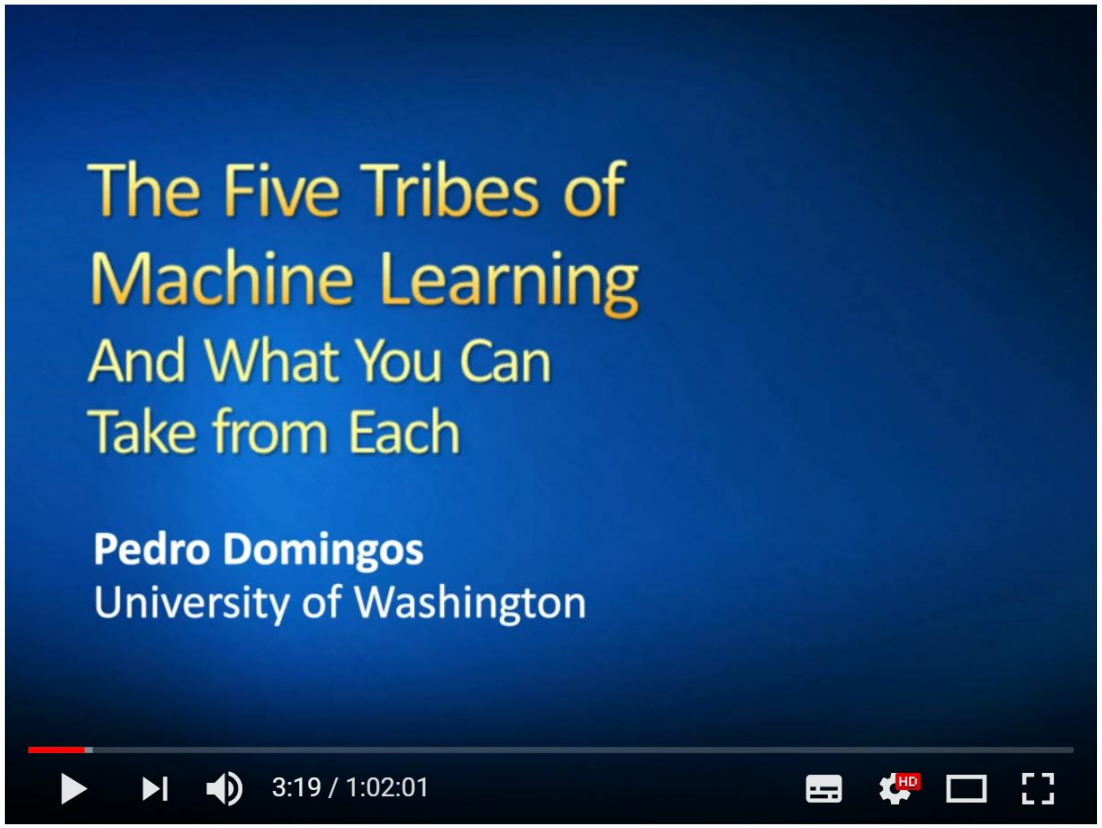


K-NN classifiers, Support vector machines, kernel machines, max-margin machines, instance-based classification, etc.

Machine learning: five tribes

Approaches	Problem Solved	Solution Origin	Solution
Symbolists	Knowledge acquisition	Logic, philosophy	*Inverse deduction*
Evolutionary	Structure discovery (nature)	Evolutionary biology	*Genetic programming*
Connectionists	Credit assignment (nurture)	Neuroscience	*Backpropagation and deep learning*
Bayesian	Uncertainty	Statistics	*Probabilistic Inference*
Analogizers	Similarity	Psychology	*Kernel machines*

Machine learning: five tribes



https://www.youtube.com/watch?v=E8rOVwKQ5-8